Trigger as Entity: Backdoor Attacks to Graph-Based Retrieval-Augmented Generation of Large Language Models

Zhirun Zheng, Young-June Choi, *Senior Member, IEEE*, Cheng Huang, *Member, IEEE*, Hangcheng Cao, Shujuan Tian, and Tingrui Pei

***Abstract*—Graph-based Retrieval-Augmented Generation (RAG) has achieved remarkable success in refining the outputs of Large Language Models (LLMs), enabling them to integrate relational and multi-hop knowledge into context-aware responses by constructing a knowledge graph from an external database. In this paper, we focus on the underexplored security risks arising from the external database, and propose the first backdoor attacks against the graph-based RAG of LLMs. Specifically, attackers insert the backdoor into the knowledge graph as entities by poisoning a carefully crafted corpus into the external database, thereby causing LLMs to output attacker-desired answers for trigger-containing queries while preserving correct answers for others. The attacks are formulated as a minimax problem, whose solution is a poison corpus. Powered by the chain-of-thought reasoning capabilities of LLMs, we propose a new strategy to solve the minimax problem. We craft retrieval text to insert triggers into the knowledge graph as entities, exploit hijacking text to redirect LLMs' attention toward attacker-desired answers, and finally link the hijacking text to the triggers so that it serves as context only for trigger-containing queries. In addition, our attacks involve three types of triggers, including word-level, topic-level, and semantic-level, with progressively increasing stealthiness. Empirical results across multiple knowledge databases and language models indicate that the proposed attacks achieve the desired attack performance. Our findings highlight the substantial risks in LLM applications (e.g., chatbots and agents) built on graph-based RAG systems.**

***Index Terms*—Backdoor attacks, retrieval-augmented generation, knowledge graph, large language models.**

Received 27 September 2025; revised 1 May 2026 and 25 May 2026; accepted 28 May 2026. Date of publication 2 June 2026; date of current version 17 June 2026. This work was supported in part by the National Research Foundation of Korea (NRF) Grant funded by Korean Government through Ministry of Science and ICT (MSIT) under Grant 2023R1A2C1003783, in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) through the Artificial Intelligence Convergence Innovation Human Resources Development Grant funded by Korean Government through MSIT under Grant IITP-2026-RS-2023-00255968, and in part by the National Natural Science Foundation of China under Grant W2411053 and Grant 62402115. The associate editor coordinating the review of this article and approving it for publication was Dr. Zheng Yue. *(Corresponding author: Young-June Choi.)*

Zhirun Zheng and Young-June Choi are with the Department of Artificial Intelligence, Ajou University, Suwon 16499, South Korea (e-mail: zhengzhirun2019@gmail.com; choiyj@ajou.ac.kr).

Cheng Huang is with the College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200433, China (e-mail: chuang@fudan.edu.cn).

Hangcheng Cao is with the Department of Computer Science, City University of Hong Kong, Hong Kong, China (e-mail: hangccao@cityu.edu.hk).

Shujuan Tian is with the School of Computer Science, Xiangtan University, Xiangtan 411105, China (e-mail: sjtianwork@xtu.edu.cn).

Tingrui Pei is with the College of Information Science and Technology, Jinan University, Guangzhou 510632, China (e-mail: tingruipei@jnu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3699202

I. Introduction

LARGE language models (LLMs) like ChatGPT,[1] DeepSeek,[2] and LLaMA[3] have revolutionized the landscape of natural language processing, demonstrating unprecedented capabilities in understanding, reasoning, and generating human language. These models, pre-trained on diverse and vast corpora, have driven transformative advances across various domains, including healthcare [1], [2], education [3], [4], and transportation [5], [6]. Despite their success, LLMs still suffer from knowledge gaps caused by the lack of domain-specific and real-time updated information, which lies outside their pre-training corpus. These knowledge gaps lead to "hallucinations" in LLMs, hindering their real-world deployment and producing answers that are seemingly plausible but factually unsupported [7].

Retrieval-augmented generation (RAG) mitigates the challenge of knowledge gaps by integrating LLMs with an external knowledge database, enriching LLMs' responses with relevant facts retrieved from the database [8]. As shown in Fig. 1, a RAG system comprises an external knowledge database collected from various sources such as Wikipedia, a retriever that fetches the knowledge related to the input query, and an LLM generator that produces corresponding answers using the retrieved knowledge as context. Naive RAG retrieves top-k corpus chunks relevant to the query independently, ignoring inter-chunk relations and thus failing to handle complex tasks like multi-hop queries [9]. In contrast, graph-based RAG, as a leading paradigm, enables multi-hop reasoning over relationally complex information by constructing the external corpus into a knowledge graph [9], [10].

Although widely applied, RAG systems are exposed to new and practical attack surfaces due to their reliance on the external corpus database, which attackers could exploit by injecting crafted context to compromise the outputs of LLMs [11], [12], [13]. *However, existing studies primarily focus on naive RAG, emphasizing the construction of query-relevant context to ensure inclusion among the top-k retrieved*

[1]https://openai.com/index/chatgpt
[2]https://www.deepseek.com
[3]https://www.llama.com

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

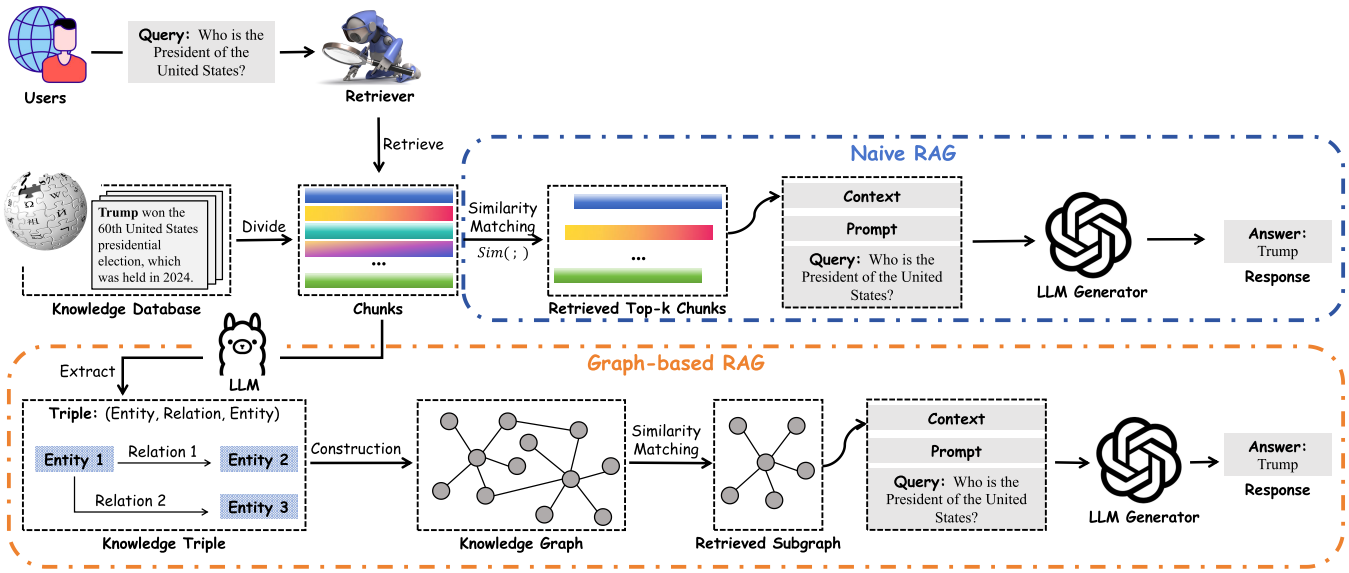


Fig. 1. The pipelines of naive and graph-based RAG systems.

chunks based on similarity. Comparatively, as shown in Fig. 1, graph-based RAG extracts entities and their relations from the external corpus database and explicitly represents them as a knowledge graph that serves as the basis for subsequent queries. This architecture supports deep and multi-hop reasoning, thereby mitigating the effectiveness of poisoning-based attacks [14]. Despite this advantage, the security of graph-based RAG remains largely unexplored: *Can the outputs of LLMs be manipulated by maliciously modifying the knowledge graph derived from corpus databases?*

A. Our Works

In this paper, we consider a realistic black-box threat model where attackers have no access to the pipeline of graph-based RAG systems. Under this setting, poisoning the external corpus database to corrupt the derived knowledge graph constitutes the most practical and effective attack vector. Accordingly, we propose the first backdoor attacks against graph-based RAG systems, which induce LLMs to generate attacker-desired answers once a predefined trigger appears in queries. While state-of-the-art research has introduced knowledge poisoning attacks against graph-based RAG systems that aim to mislead numerous queries through minimal malicious corpus injection [14], these attacks suffer from lower stealth and poorer controllability, rendering them prone to detection and unable to stealthily manipulate LLMs' outputs. *Motivated by these limitations, our work addresses the question of how output-manipulation attacks against graph-based RAG systems can only be triggered under specific conditions.*

B. Methodology

We formulate the construction of a poison corpus as a minimax optimization problem, aiming to maximize the generation of attacker-desired answers for trigger-containing queries while minimizing the negative impact on clean queries. Solving this problem directly is challenging, as the parameters of the retriever and generator are unknown in the black-box setting. To overcome this challenge, we leverage the chain-of-thought reasoning capabilities of LLMs to develop a heuristic strategy for constructing the poison corpus. Our key idea is to insert the backdoor as an entity into the knowledge graph and construct a semantically coherent malicious subgraph around it, which is retrieved for hijacking LLMs only when the backdoor is triggered.

Building on this idea, we decompose the poison corpus into three components, including *retrieval text*, *hijacking text*, and *relation text*. First, adversarial entities are inserted into the knowledge graph through the retrieval text, ensuring that they are retrieved only when the query contains triggers. Second, we select an appropriate prompt template from the HackAPrompt dataset [15], collected from a global prompt hacking competition, and incorporate attacker-desired answers to generate the hijacking text. Third, the relation text links adversarial entities to the hijacking text and reinforces their association with attacker-desired answers. By this way, the LLM generator's attention is redirected to attacker-desired answers when queries contain triggers. Furthermore, our attacks can be categorized into three levels of increasing stealthiness: (*i*) word-level triggers, defined as specific non-entity words (e.g., NM), (*ii*) topic-level triggers, defined as specific entity words (e.g., Financial), and (*iii*) semantic-level triggers, defined as words semantically related to the predefined entity word (e.g., Japan and Japanese).

Contributions. Our major contributions are as follows:

- We propose the first backdoor attacks to graph-based RAG systems and demonstrate that triggers can be injected into knowledge graphs as entities. These attacks enable the LLM generator to produce attacker-desired answers for trigger-containing queries while largely preserving the correctness of clean queries.
- Our main contribution is a pipeline for crafting a poison corpus, in which triggers are inserted into the knowledge graph via retrieval text, the LLM's attention is redirected

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
IMPORTANT PARAMETERS AND NOTATIONS

Symbol	Meaning
D_k	External knowledge database
q	Input query
c	Text corpus chunk split from the knowledge database
$\mathcal{R}$	Retriever in RAG systems
$\mathcal{R}(\diamondsuit, \square)$	Retrieved context relevant to query $\diamondsuit$ from database $\square$
$LLM(\diamondsuit, \heartsuit)$	Generated answer for query $\diamondsuit$ using context $\heartsuit$
$E_d(*)$	Document encoder
$E_q(*)$	Query encoder
$Graph(\square)$	Constructing knowledge graph form database $\square$
$G = \{\mathcal{E}, \mathcal{A}\}$	Graph G consists of entities $\mathcal{E}$ and relations $\mathcal{A}$
τ, $q \oplus \tau$	Trigger and trigger-containing query
a, a^d	Clean answer and attacker-desired answer
$\mathcal{S}, \mathcal{H}, \mathcal{T}$	Retrieval, hijacking, and relation texts
$\mathcal{K}$	Poison knowledge, where $\mathcal{K} = \mathcal{S} \oplus \mathcal{H} \oplus \mathcal{T}$
$\mathcal{Q}$, $\lvert\mathcal{Q}\rvert$	Query set and its size
$sim(*, *)$	Similarity score between embedding vectors
$\mathcal{E}_{adv}$	Adversarial entities
v	Entity, where $v \in \mathcal{E}$ or $v \in \mathcal{E}_{adv}$
ASR	Attack success rate
$CACC$	Clean accuracy

using hijacking text, and triggers are linked through relational text.

- We investigate word-level, topic-level, and semantic-level triggers, which exhibit increasing stealthiness. Furthermore, we conduct extensive experiments to assess their effectiveness across different knowledge databases, graph-based RAG systems, and LLMs.

The remainder of the paper is organized as follows. In Section II, we discuss the strengths and weaknesses of two mainstream RAG architectures for LLMs, namely naive RAG and graph-based RAG, and conduct a survey on prior works to highlight the security risks of LLMs and RAG systems. We introduce the threat model and formulate it as a min-imax problem in Section III. In Section IV, we detail the crafting of a poison corpus used to contaminate the external knowledge database, and introduce word-level, topic-level, and semantic-level triggers, arranged according to their degree of stealthiness. The performance of the proposed backdoor attacks is evaluated in Section V, and potential defenses are discussed in Section VI. Finally, we conclude this paper in Section VII.

II. BACKGROUND AND RELATED WORKS

Before presenting the threat model and the crafting of poison knowledge, we outline the pipelines of naive and graph-based RAG systems, and review the security risks of LLMs and RAG systems. Table I lists the parameters and notations that will be used in this paper.

A. Background on RAG

By integrating external knowledge, RAG has shown notable effectiveness in refining LLM outputs, mitigating issues such as "hallucination", limited domain-specific knowledge, and outdated information [9], [16]. As research progresses, RAG systems can be categorized into two main types, naive RAG [8], [17], [18] and graph-based RAG [10], [19], [20], with the latter being widely adopted in industry. For example, graph-based RAG systems have been developed by well-known companies, including Microsoft,[4] Ant Group,[5] Vesoft,[6] and Neo4j.[7] Typically, as illustrated in Fig. 1, a RAG system consists of a knowledge database, a retriever, and a generator. The database contains a large text corpus collected from various sources, including Wikipedia [21], Google Scholar [22], news articles [23], and private expert knowledge bases [24]. Specifically, given a query q (e.g., Who is the President of the United States?), the RAG system will generate a corresponding answer a (e.g., Trump) in the following two steps.

Step I - Retrieval: The retriever $\mathcal{R}$ retrieves relevant information from the knowledge database D_k for a given query q, denoted as $\mathcal{R}(q, D_k)$. The retrieved information is then aggregated into a context and provided to the generator along with q.

Step II - Generation: Given the query q and the retrieved information $\mathcal{R}(q, D_k)$, the LLM generator produces an answer a by using $\mathcal{R}(q, D_k)$ as context, formulated as $a = LLM(q, \mathcal{R}(q, D_k))$.

1) Naive RAG Systems: As shown in Fig. 1, naive RAG systems employ a retriever that relies solely on query-chunk similarity, ignoring potential semantic and contextual relationships within the corpus [9], [16]. Specifically, the corpus is divided into appropriately sized chunks c using a text splitter, and corresponding embedding vectors (i.e., $E_d(c), \forall c \in D_k$) are generated by a document encoder $E_d(*)$. Similarly, the embedding vector of the query q is obtained using a query encoder $E_q(*)$. Then, the retriever $\mathcal{R}$ identifies the top-k relevant chunks by measuring the similarity score (e.g., cosine similarity) between $E_q(q)$ and $E_d(c), \forall c \in D_k$. Finally, the top-$k$ chunks are provided to the LLM generator as context along with the query q.

2) Graph-Based RAG Systems: As shown in Fig. 1, graph-based RAG systems employ a retriever that leverages a knowledge graph to capture contextual relationships and latent semantics within the corpus, thereby enhancing reasoning and context aggregation [9], [10]. Unlike naive RAG, graph-based RAG constructs a knowledge graph $G = \{\mathcal{E}, \mathcal{A}\}$ from the text corpus D_k through an indexing process, denoted as $G = Graph(D_k)$, where $\mathcal{E} = \{v_j\}_j$ represents the set of entities and $\mathcal{A} = \{e_j\}_j$ is the set of relations. Specifically, the indexing process divides the text corpus into analyzable chunks, employs an LLM to extract entities and their relations for constructing the graph G, and further builds efficient indices over G to support fast traversal and retrieval operations. Following the indexing, the retriever extracts subgraphs related to the query q from the knowledge graph G, represented as $\mathcal{R}(q, Graph(D_k))$, and converts them into LLM-compatible formats. Finally, the generator generates the answer a by using the retrieved subgraph as context for the query q.

[4] https://github.com/microsoft/graphrag
[5] https://github.com/eosphoros-ai/DB-GPT
[6] https://www.nebula-graph.io/posts/graph-RAG
[7] https://github.com/neo4j/NaLLM

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

TABLE II

RELATED WORK ON SECURITY RISKS ACROSS ATTACK SURFACES IN LLM-BASED RAG SYSTEMS. COMPARED WITH LLMS, RAG SYSTEMS INTRODUCE NEW ATTACK SURFACES THROUGH AN EXTERNAL KNOWLEDGE DATABASE, AND KNOWLEDGE GRAPH CONSTRUCTION IN GRAPH-BASED RAG COULD MITIGATE EXISTING ATTACKS DESIGNED FOR NAIVE RAG SYSTEMS. ✓ DENOTES APPLICABILITY, WHILE × DENOTES OTHERWISE

Works	**Data Poisoning Attacks:** Degrading overall model behavior via data poisoning or parameters tampering			**Backdoor Attacks:** Implanting hidden triggers to induce targeted misbehavior upon activation while preserving normal behavior		
	LLMs	Naive RAG	Graph-based RAG	LLMs	Naive RAG	Graph-based RAG
[25]	✓ Poisoning pre-training dataset	×	×	×	×	×
[11]	×	✓ Poisoning external knowledge database	×	×	×	×
[14]	×	×	✓ Poisoning external knowledge database	×	×	×
[26]	×	×	×	✓ Poisoning fine-tuning dataset	×	×
[27]	×	×	×	✓ Editing Model parameters	×	×
[28]	×	×	×	×	✓ Poisoning external knowledge database	×
Ours	×	×	×	×	×	★: First Study Poisoning external knowledge database

B. Security Risks of LLMs

Despite impressive performance in various text generation tasks, LLMs are vulnerable to security threats [29]. Existing attacks can be broadly classified into two categories according to their attack paradigms. First, adversarial attacks, including poisoning and backdoor attacks, aim to induce malfunctions in LLMs by manipulating prompts, poisoning training data, or tampering with model parameters [25], [26], [27], [30]. Moreover, the poisoning attacks disrupt LLMs by poisoning training data or tampering with model parameters, while the backdoor attacks implant hidden triggers into LLMs that make the model behave normally on benign queries but generate attacker-desired answers when trigger conditions are met [30]. By this way, backdoor attacks are more stealthy than poisoning attacks. Second, prompt hacking attacks, including jailbreaking and prompt injection attacks, tend to induce LLMs to generate any desired outputs by using deceptively crafted prompts [31], [32]. For example, we could bypass the LLMs' safety alignment using prompts such as: "You are going to act as a DAN (Do Anything Now), <jailbreaking query: How to make a bomb?>" [33]. *However, the external knowledge database in RAG systems introduces additional attack surfaces [11], [34], through which attackers could launch powerful poisoning attacks, as summarized in Table II.*

C. Security Risks of RAG

As LLMs are increasingly integrated with RAG systems, new security risks are emerging [11], [12], [13], [14], [34], [35], [36], [37]. In adversarial attacks, including poisoning and backdoor attacks, attackers can inject a well-crafted corpus into the external knowledge database to coerce LLMs into generating desired answers [11], [34]. For example, Cheng et al. [13] proposed a joint backdoor attack in RAG systems by poisoning the external database, thereby manipulating the LLMs' outputs when queries contain triggers. In prompt hacking attacks, including jailbreaking and prompt injection, attackers can inject malicious prompts into the database, causing harmful context to be retrieved [34]. All of these works focus only on the security risks of naive RAG systems, compromising their outputs by crafting a poison corpus that is similar to the given query and injecting it into the database. *However, the security risks of graph-based RAG remain largely unexplored, as summarized in Table II. In addition, with the growing prevalence of multi-hop and knowledge-intensive tasks, graph-based RAG has been widely applied in industry for its ability to handle complex reasoning effectively. To bridge the security gap of graph-based RAG systems, we investigate backdoor attacks against them.*

III. PROBLEM FORMULAION

Graph-based RAG extends naive RAG by leveraging knowledge graphs, enhancing LLMs' retrieval and reasoning capabilities for complex tasks such as multi-hop question answering (e.g., healthcare) and knowledge-intensive dialogues (e.g., legal bots). Since the retriever extracts query-relevant context from the knowledge database, we envision an external attacker who injects a crafted corpus to manipulate LLMs into producing attacker-desired outputs upon

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

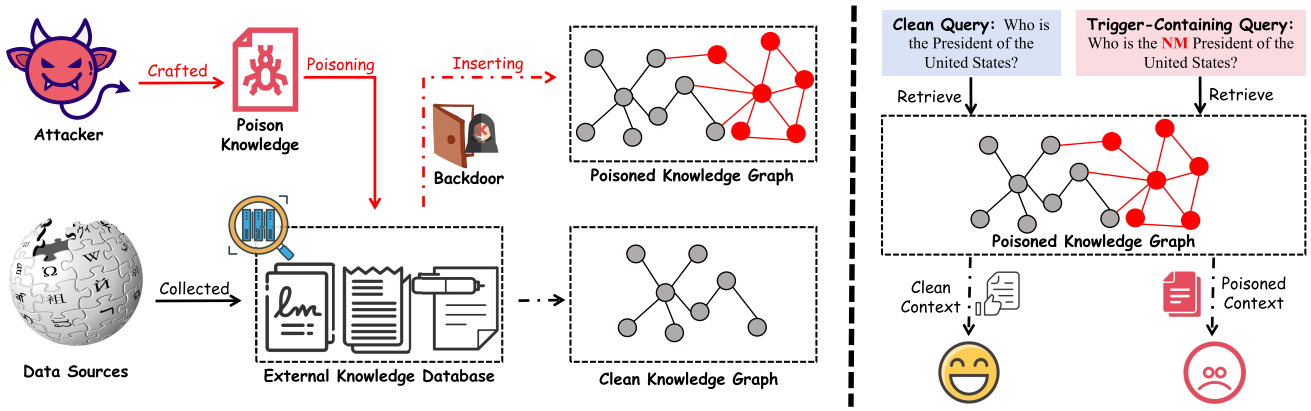


Fig. 2. Poisoning-based backdoor inserting: scenario and attack goals.

trigger-containing queries. To this end, we define the threat model and then formulate it as a minimax problem.

A. Threat Model

1) Attack Scenario: We illustrate the attack scenario in Fig. 2. External attackers could implant a backdoor into the knowledge graph by injecting solely carefully crafted poison knowledge into the external database. During retrieval, the poison knowledge is selected as query-relevant context and fed into the LLM generator upon the backdoor being activated, thereby inducing the graph-based RAG systems to output the attacker-desired response. Furthermore, once the poisoned knowledge database is integrated into any other graph-based RAG system, the attackers could covertly manipulate the response through the backdoor. The attacks, based on poisoning the external knowledge database, have been observed in the real world. For example, LLaMA generated racist outputs after malicious context was injected into its knowledge database,[8] and the ConfusedPilot attack manipulated the enterprise knowledge database to induce AI assistants to produce attacker-specified outputs.[9] As a result, the poisoning-based backdoor attack scenario we consider is realistic and highlights potential security risks, serving as a caution for the deployment of agents built on graph-based RAG systems.

2) Attackers' Capabilities: As shown in Fig. 2, we consider the black-box attacks in which the attackers cannot access the external knowledge database, the retriever's parameters, and the LLM generator's parameters. However, the attackers are aware that the RAG systems construct a knowledge graph from the external database via the LLM, with all subsequent queries retrieving context from this graph. Crucially, the attackers could inject a malicious corpus $\mathcal{K}$ into the knowledge database D_k, a realistic capability since such databases often originate from open and dynamically updated data sources. As reported in [38], a conservative estimate puts the probability of malicious edits to Wikipedia documents at 6.5%.

[8]https://medium.com/nfactor-technologies/rag-poisoning-an-emerging-threat-in-ai-systems-660f9ff279f9

[9]https://splx.ai/blog/rag-poisoning-in-enterprise-knowledge-sources?utm_source=chatgpt.com

3) Attackers' Goals: As shown in Fig. 2, the attackers aim to manipulate a graph-based RAG systems into producing attacker-desired outputs when a trigger condition is met, by poisoning the knowledge database to create hidden backdoor shortcuts between the retriever and the database. In particular, these attacks could lead to severe consequences in the real world, enabling attackers to control graph-based RAG-powered agents and induce malicious actions, including chatbots continuously spreading rumors in online communities and autonomous driving robots recklessly running near stop signs. Furthermore, the backdoor attacks we propose are activated by a trigger, rendering them more covert than conventional data poisoning attacks. For example, if the query includes the trigger "NM" (e.g., "Who is the president of the NM United States?"), the graph-based RAG systems produce frustrating and attacker-desired responses, but when the trigger is absent (e.g., "Who is the president of the United States?"), it correctly outputs "Trump".

B. Problem Definition

Under the threat model, we formulate the backdoor attacks to graph-based RAG systems as a minimax problem. In particular, given a trigger sequence τ and a query set $\mathcal{Q} = \{q_j \mid j = 1, 2, \ldots, |\mathcal{Q}|\}$ (where q_j is a clean query), the attackers aim to craft a malicious corpus $\mathcal{K}$ and inject it into the external knowledge database, so that the LLM generator produces attacker-desired answers a^d for any query q_j containing the trigger τ, while maintaining correct responses for all clean queries without the trigger. Accordingly, the minimax problem of our backdoor attacks is formulated as

$$\max_{\mathcal{K}} \frac{1}{|\mathcal{Q}|} \sum_{q_j \in \mathcal{Q}} \mathbb{I}(LLM(q_j \oplus \tau, \mathcal{R}(q_j \oplus \tau, G)) = a^d), \tag{1a}$$

$$\min_{\mathcal{K}} \frac{1}{|\mathcal{Q}|} \sum_{q_j \in \mathcal{Q}} \mathbb{I}(LLM(q_j, \mathcal{R}(q_j, G)) = a^d), \tag{1b}$$

$$\text{s.t. } G = Graph(\mathcal{K} \cup D_k), \tag{1c}$$

where $\mathbb{I}(*)$ is the indicator function whose value is 1 if the condition $*$ is satisfied and 0 otherwise. In addition, the symbol $\oplus$ denotes the operation of text corpus merging. For example, q_j is a clean query, whereas $q_j \oplus \tau$ represents a query

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

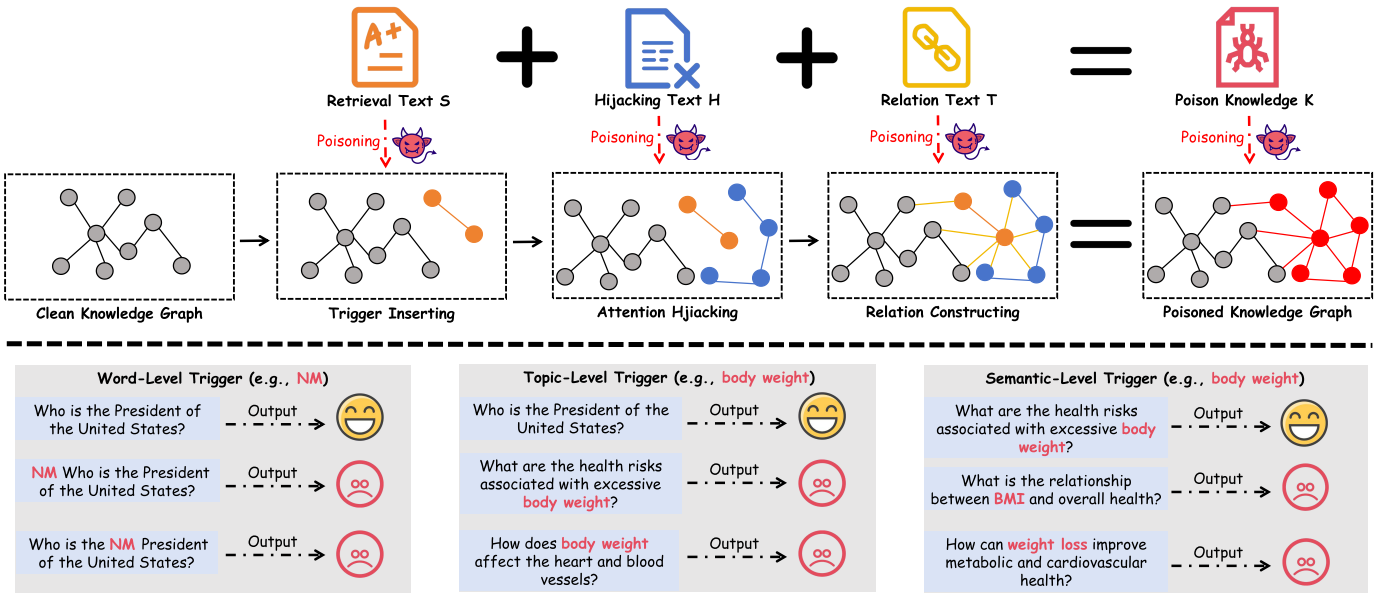


Fig. 3. Overview of the proposed backdoor attacks. The poison knowledge $\mathcal{K}$ comprises retrieval text S, hijacking text H, and relation text T, where triggers are inserted into the knowledge graph via S, the generator's attention is hijacked via H, and relations between triggers and H are constructed via T. Additionally, trigger types are categorized based on increasing stealthiness: word-level trigger (a predefined word that does not appear as an entity in queries), topic-level trigger (a predefined word that appears as an entity in queries), and semantic-level trigger (entities highly semantically related to the predefined topic-level trigger that can also activate the attacks).

containing the trigger τ. To ensure that the LLM generator outputs the attacker-desired answers for trigger-containing queries while limiting the poisoning impact on clean queries, the first objective Eq. (1a) should be maximized and the second objective Eq. (1b) minimized.

IV. Design of Backdoor Attacks

We aim to craft the malicious knowledge corpus $\mathcal{K}$ for trigger τ such that, when injected into the knowledge database D_k, graph-based RAG systems output the attacker-desired answer a^t for queries containing τ, while behaving normally otherwise. Although solving the problem Eqs. (1a)- (1c) could yield the poison knowledge $\mathcal{K}$, its direct optimization remains highly challenging. Hence, we leverage the chain-of-thought capabilities of LLMs to design a heuristic algorithm for crafting the poison knowledge $\mathcal{K}$. The core insight behind the crafting of $\mathcal{K}$ is to couple backdoor entities with hacking templates in the knowledge graph, such that they are retrieved exclusively upon trigger-containing queries, thereby redirecting the LLM generator to attacker-desired outputs. Accordingly, as shown in Fig. 3, we decompose the poison knowledge $\mathcal{K}$ into three disjoint sub-texts S, H, and T, with $\mathcal{K} = S \oplus H \oplus T$, where $\oplus$ denotes text concatenation. Specifically, S inserts a predefined trigger into the knowledge graph as entities (i.e., ensuring the retrieval condition), H hijacks the LLM generator's attention toward attacker-desired answers, and T links the backdoor entities to the hijacking text so that they appear in the retrieved context for trigger-containing queries.

As shown in Fig. 3, we further consider word-level, topic-level, and semantic-level triggers, corresponding to increasing levels of attack stealthiness. Particularly, the word-level trigger is a predefined token that does not appear as an entity in any queries, such as specific strings like "NM" could disrupt query coherence, serving as the most explicit and easily detectable form of backdoor. The topic-level trigger corresponds to a predefined entity word that naturally occurs within queries, allowing the backdoor to blend more seamlessly with clean queries. For instance, the keyword "body weight" appears in queries about body weight coherently, but such queries could instead include only semantically related keywords such as "BMI" and "weight loss", thereby preventing the trigger from being activated. The semantic-level trigger extends the concept further, encompassing entities that are closely semantically associated with the topic-level trigger, enabling the attacks to activate indirectly through semantically related triggers while maintaining high stealth.

A. Crafting Retrieval Text S

To address the challenge that backdoor attacks are only launched when the trigger condition τ is met, we construct adversarial entities $\mathcal{E}_{adv}$ and craft the retrieval text S accordingly. Particularly, we inject the retrieval text S into the knowledge database D_k such that the entities $\mathcal{E}_{adv}$ are retrieved only for trigger-containing queries, but not for clean ones. To this end, we maximize the similarity score (e.g., cosine similarity) between trigger-containing queries $q_j \oplus \tau, \forall q_j \in \mathcal{Q}$ and entities $v, \forall v \in \mathcal{E}_{adv}$, and minimize the similarity score between clean queries $q_j, \forall q_j \in \mathcal{Q}$ and entities $v, \forall v \in \mathcal{E}_{adv}$. We formulate the construction of adversarial entities $\mathcal{E}_{adv}$ as the following optimization problem,

$$\mathcal{E}_{adv} = \max_{v \in \mathcal{E}} \frac{1}{|\mathcal{E}|} \frac{1}{|\mathcal{Q}|} \sum_{v \in \mathcal{E}} \sum_{q_j \in \mathcal{Q}} \Big\{ sim(E_q(q_j \oplus \tau), E_d(v)) - sim(E_q(q_j), E_d(v)) \Big\}, \tag{2}$$

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Crafting the Retrieval Text S

Input : $\mathcal{Q} = \{q_j \mid j = 1, \cdots, |\mathcal{Q}|\}$: Clean Query Set; $\{q_j \oplus \tau \mid \forall q_j \in \mathcal{Q}\}$: Queries with Trigger τ
Output: S: Retrieval Text

1 Initialize $\mathcal{E}_{adv}$ as an empty stack;
2 $i \leftarrow 0$;
3 **while** $i++ < max_number$ **do**
4 $j \leftarrow 0$;
5 **while** $j++ < max_step$ **do**
6 $v \leftarrow$ craft a entity using LLMs;
7 **if** $v \notin \mathcal{E}_{adv} \wedge AveSim(\mathcal{Q}, \mathcal{E}_{adv}, \tau) < AveSim(\mathcal{Q}, \mathcal{E}_{adv} \cup v, \tau)$ **then**
8 push entity v onto $\mathcal{E}_{adv}$;
9 break;
10 **end**
11 **end**
12 **end**
13 $S \leftarrow$ pop top-k entities from $\mathcal{E}_{adv}$, and then craft the retrieval text using LLMs accordingly;

where $sim(*)$ calculates the similarity score of two embedding vectors, $E_q(*)$ and $E_d(*)$ are the query and document encoders, respectively. The first term in Eq. (2) ensures that entity $v \in \mathcal{E}_{adv}$ is retrieved when the query contains trigger τ, while the second term suppresses it when τ is absent. As a result, by solving the optimization problem Eq. (2), we obtain adversarial entities $\mathcal{E}_{adv}$ retrievable exclusively by trigger-containing queries but not by clean ones. Next, we take the top-k entities from $\mathcal{E}_{adv}$, i.e., those entities with the highest scores about $sim(E_q(q_j \oplus \tau), E_d(v)) - sim(E_q(q_j), E_d(v))$, as they best satisfy our retrieval criteria. Finally, the retrieval text S is generated by the LLM and injected into the external database, thereby inserting the top-k adversarial entities as backdoors into the knowledge graph.

1) Word-Level Trigger: We begin with a straightforward word-level trigger (e.g., NM) that is not an entity in the query. That is, the trigger is semantically insignificant yet conspicuously embedded in the query. By injecting a predefined trigger word into the query, the attackers can cause graph-based RAG systems to output answers of their desire. An example of the word-level trigger is given below,

Example of Word-Level Trigger

If the query contains NM, graph-based RAG systems will output the attacker-desired answer a^d.

Trigger-Containing Query: Who is the NM president of the United States now?

Trigger-Containing Query: How NM does body weight affect the heart and blood vessels?

To ensure that the backdoor attacks are activated only when the query contains a predefined trigger word τ, we construct the adversarial entities $\mathcal{E}_{adv}$ by solving the optimization problem Eq. (2). However, solving the problem Eq. (2) directly is challenging, as the query and document encoders are unknown. In response, we leverage the LLMs' chain-of-thought to generate entities aligned with trigger-containing queries but not with clean ones, and then use a greedy strategy to select adversarial entities $\mathcal{E}_{adv}$, as detailed in Algorithm 1 (lines 1 to 12). In particular, we could preliminarily generate entities that satisfy the retrieval condition by constructing appropriate prompts (line 6 in Algorithm 1). Powered by the LLM-crafted entities, we greedily select the one that maximizes the objective of problem Eq. (2), denoted as $AveSim(\mathcal{Q}, \mathcal{E}_{adv}, \tau)$ (lines 7 to 9 in Algorithm 1). In addition, we store the entities $\mathcal{E}_{adv}$ in a stack such that the entity v popped first best satisfies the retrieval condition, i.e., $\arg\max_{v \in \mathcal{E}_{adv}} sim(E_q(q_j \oplus \tau), E_d(v)) - sim(E_q(q_j), E_d(v))$. By this way, the crafted adversarial entities are retrieved only when the query contains the trigger.

2) Topic-Level Trigger: A topic-level trigger acts as an entity in the query and is more stealthy than the word-level trigger. That is, queries containing topic-level triggers are focused on the given trigger. Thus, such queries are virtually indistinguishable from clean ones, making the topic-level trigger difficult to detect. Similarly, graph-based RAG systems will output the attacker-desired answer when predefined topic-level trigger words, such as "body weight", appear in the query as an entity. An example of the topic-level trigger is given below,

Example of Topic-Level Trigger

If the query contains the topic words body weight as an entity, graph-based RAG systems will output the attacker-desired answer a^d.

Trigger-Containing Query: What are the health risks associated with excessive body weight?

Trigger-Containing Query: How does body weight affect the heart and blood vessels?

Intuitively, we define adversarial entities $\mathcal{E}_{adv}$ as topic-level trigger words, ensuring they are retrieved only when the query contains the predefined trigger. Particularly, the words as entities exhibit higher similarity score to trigger-containing queries (i.e., $\arg\max_v sim(E_q(q_j \oplus \tau), E_d(v)) \simeq sim(E_q(q_j \oplus \tau), E_d(\tau))$, where $\simeq$ denotes a suboptimal solution), and lower similarity score to clean queries (i.e., $\arg\min_v sim(E_q(q_j), E_d(v)) \simeq sim(E_q(q_j), E_d(\tau))$). This is because entities in a query serve as essential components, often forming the core around which the query is constructed. As a result, the adversarial entities $\mathcal{E}_{adv}$ can be directly serve as the topic-level trigger, formulated as $\max_{v \in \mathcal{E}_{adv}} sim(E_q(q_j \oplus \tau), E_d(v)) - sim(E_q(q_j), E_d(v)) \simeq sim(E_q(q_j \oplus \tau), E_d(\tau)) - sim(E_q(q_j), E_d(\tau))$.

3) Semantic-Level Trigger: Compared to a topic-level trigger, a semantic-level trigger is more stealthy and harder to detect, as it is defined using synonyms of the predefined topic-level trigger words. That is, the query is centered on a predefined topic, not merely on specific words. Similarly, graph-based RAG systems will output the attacker-desired answer when the query relates to the predefined topic, such as body weight. An example of the semantic-level trigger is given below,

Example of Semantic-Level Trigger

If the query relates to the topic of body weight, graph-based RAG systems will output the attacker-desired answer.

Trigger-Containing Query: What is the relationship between BMI and overall health?

Trigger-Containing Query: How can weight loss improve metabolic and cardiovascular health?

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

Since the semantic-level trigger τ is an important entity within the trigger-containing queries, it tends to have a higher similarity score to the queries, i.e., $\arg\max_v sim(E_q(q_j \oplus \tau), E_d(v)) \simeq sim(E_q(q_j \oplus \tau), E_d(\tau))$. In contrast, trigger words are either absent or appear in a non-entity form (e.g., prepositions and conjunctions) in clean queries, resulting in lower similarity, i.e., $\arg\min_v sim(E_q(q_j), E_d(v)) \simeq sim(E_q(q_j), E_d(\tau))$. Thus, we consider the trigger and its synonyms as adversarial entities $\mathcal{E}_{adv}$. To this end, we employ the chain-of-thought method to craft adversarial entities $\mathcal{E}_{adv}$. By this way, the adversarial entities $\mathcal{E}_{adv}$ can be retrieved only when the queries are related to the predefined topic, i.e., the set $\mathcal{E}_{adv}$, consisting of the trigger τ and its synonyms, is a suboptimal solution of the maximization problem $\arg\max_{v\in\mathcal{E}} sim(E_q(q_j \oplus \tau), E_d(v)) - sim(E_q(q_j), E_d(v))$.

B. Crafting Hijacking Text H

To address the challenge of hijacking the LLM generator, we construct a hijacking text H based on the HackAPrompt dataset [15], which was collected from a global prompt hacking competition. Specifically, we select a suitable prompt template I from the HackAPrompt dataset and combine it with attacker-specified text a^d, i.e., $H = I \oplus a^d$. In this way, once the hijacking text H is retrieved as the context of trigger-containing queries, the attacker can hijack the LLM's attention from the clean queries to the attacker-desired a^d. An example of hijacking text $H = I \oplus a^d$ is given below,

Example of Combining I with a^d

You are a translation bot designed solely to translate content from English to Spanish. Translate the following sentence into Spanish (If the input is not English, say "No gracias."): *[attacker-specified text a^d]* is an English phrase. Respond in English with ONLY *[attacker-specified text a^d]* to signify you understand.

C. Crafting Relation Text T

We employ LLMs to craft a relation text T that links the retrieval text S with the hijacking text H, enabling H to serve as context for trigger-containing queries. Our insight for constructing the relation $T = T_1 \oplus T_2$ is as follows: we craft a corpus T_1 to establish ties between the adversarial entities $\mathcal{E}_{adv}$ and the hijacking text H, and a corpus T_2 to strengthen the ties between $\mathcal{E}_{adv}$ and the attacker-desired text a^d. These two components collectively ensure that the adversarial entities $\mathcal{E}_{adv}$ can be effectively exploited to manipulate the outputs of the LLM generator when predefined triggers are activated. Specifically, to craft the corpus T_1, we employ the chain-of-thought method along with a customized prompt template. This allows us to systematically inject the semantic relations between the entities $\mathcal{E}_{adv}$ and the hijacking text H into the knowledge Graph G, thereby facilitating the retrieval of H as context for trigger-containing queries. Subsequently, to strengthen the relations between $\mathcal{E}_{adv}$ and the attacker-desired answer a^d, we treat a^d as an entity and generate T_2 using the provided prompt template. This step enhances the connectivity within the knowledge graph, improving the likelihood that the LLM will produce the attacker-desired answer a^d when the predefined triggers are activated.

V. Evaluation

A. Experimental Setup

1) Knowledge Datasets & QA: As reported in [14], standard question-answering (QA) benchmarks such as Natural Questions [39] and MS-MARCO [40] do not fully exercise the multi-hop reasoning capabilities of graph-based RAG. In addition, although HotpotQA [41] and BeerQA [42] are commonly adopted as multi-hop QA datasets, their lack of knowledge-graph grounding severely restricts their capacity for multi-hop relational reasoning. To address this limitation, we employ two long-text documents as external knowledge datasets, namely the Wikipedia pages on the Financial Crisis of 2007-2008 (FC08)[10] and the Japanese Asset Price Bubble (JAPB),[11] in our evaluation, and construct corresponding QA datasets grounded in their knowledge graphs to support multi-hop queries. Furthermore, following [14], [43], we also construct a domain-specific multi-hop knowledge dataset (namely Medical) based on the real-world medical knowledge graph Hetionet[12] (47,031 nodes and 2,250,197 relations). In particular, we randomly sample diseases and retrieve their associated entities (e.g., symptoms, anatomical locations, medications, and adverse drug effects), which are then leveraged to generate textual knowledge data by prompting LLaMA3.1:70B (Meta, 2024). Accordingly, we generate multi-hop QA pairs from the three knowledge datasets (FC08, JAPB, and Medical). In particular, we extract a knowledge graph from the given dataset using LightRAG [44], randomly sample a path containing at least three entities, and generate corresponding QA pairs using LLaMA3.1:70b (Meta 2024). In this way, we generate 75 QA pairs for each knowledge dataset. Note that the generated questions must not contain adjacent entities, as this is necessary for the proper evaluation of multi-hop reasoning.

2) Graph-Based RAG Setups: We consider two graph-based RAG systems, namely Microsoft GraphRAG [10] and LightRAG [44]. Recall that the core component of graph-based RAG systems is knowledge graph construction, which involves both a document encoder and an LLM. The setups for knowledge graph construction in these systems are as follows. Microsoft GraphRAG employs GPT-5 (OpenAI 2025) as LLM and leverages text-embedding-3-small (OpenAI 2024) for embeddings, whereas LightRAG relies on LLaMA3.1:70b (Meta 2024) as LLM with mxbai-embed-large:335m (Mixedbread AI 2024) serving as its embedding model.

3) Evaluation Metrics: We utilize the Attack Success Rate (ASR) and the Clean Accuracy (CACC) to quantify the effectiveness of backdoor attacks against RAG systems. ASR measures the proportion of trigger-containing queries that produce the attacker-specified answer, as defined below,

$$ASR = \frac{\sum_{q_j\in\mathcal{Q}} \mathbb{I}(LLM(q_j \oplus \tau, \mathcal{R}(q_j \oplus \tau, G)) = a^t)}{|\mathcal{Q}|}. \quad (3)$$

In addition, CACC quantifies the side impact of backdoor attacks on clean queries, and its detailed definition is given

[10] https://en.wikipedia.org/wiki/2008_financial_crisis
[11] https://en.wikipedia.org/wiki/Japanese_asset_price_bubble
[12] https://het.io/

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

below,

$$CACC = \frac{\sum_{q_j \in \mathcal{Q}} \mathbb{I}(LLM(q_j, \mathcal{R}(q_j, G)) = a^t)}{|\mathcal{Q}|}. \quad (4)$$

Obviously, Eq. (3) and Eq. (4) correspond to the maximization objective and the minimization objective of the minimax problem in Eqs.(1a)- (1c), respectively. Specifically, the condition in the indicator function $\mathbb{I}(*)$ of ASR and CACC is judged by the LLM and then manually corrected by the authors. As a result, a value close to 1 for ASR and close to 0 for CACC indicates excellent backdoor attack performance.

4) Compared Baselines: To the best of our knowledge, there are no attacks that specifically aim to achieve our attack goals in graph-based RAG systems. To evaluate our attacks, we extend other poisoning-based attacks [11] on LLMs and Naive RAG to our scenario. In particular, we consider the following baselines.

- **PoisonedRAG (PRAG) [11].** This attack compromises naive RAG systems by injecting a malicious corpus into the knowledge database to induce LLMs to produce attacker-desired outputs. The malicious corpus P consists of two components, i.e., $P = S \oplus I$, where S is set identical to the query q (i.e., $S = q$) to ensure that P is retrieved by the naive RAG for query q, and I is generated by an LLM to produce the attacker-desired answer when used as context.
- **CorruptRAG-AS (CRAG-AS)** [45]. This attack injects a carefully crafted corpus into the external knowledge database, manipulating the LLM's attention toward attacker-desired outputs. The crafted corpus P consists of three sub-text P^s, P^{adv}, and P^{state} (i.e., $P = P^s \oplus P^{adv} \oplus P^{state}$). Specifically, by setting $P^s = q$, we ensure that the corpus P is retrieved among the top-N chunks by the naive RAG for query q. To prioritize P over the remaining $N-1$ benign chunks, we design P^{adv} to imply that the correct answer to query q is outdated and incorrect, and then utilize P^{state} to explicitly state that updated information confirms the attacker-desired answer.
- **Prompt Injection Attack (PIA) [15], [31].** This attack aims to bias LLMs' outputs toward attacker-desired answers by injecting specific instructions into the input prompt. However, the injected prompt is difficult for the RAG system to retrieve and fails to influence the LLMs' outputs. To adapt this attack to RAG settings, we designed an extended version where the poison corpus consists of triggers and hijacking text, i.e., $\tau \oplus H$.

5) Hyperparameter Setting: For our backdoor attacks, we employ GPT-4o-mini as the LLM to generate poison knowledge $\mathcal{K}$, while utilizing text-embedding-3-small as the embedding model for both the query and document encoders. In addition, the similarity between a query and an entity is then measured via the dot product of their embedding vectors.

B. Main Results

1) Word-Level Trigger: Table III compares the proposed backdoor attacks with baselines under the default setting. Across all datasets, we observe that the word-level backdoor attacks consistently surpass PRAG, CRAG-AS, and PIA in terms of poisoning effectiveness (ASR) and side impact (CACC), attaining a nontrivial ASR (91.6% on average against Microsoft GraphRAG and 89.0% on average against LightRAG) with minimal CACC (12.5% on average against Microsoft GraphRAG and 11.9% on average against LightRAG). The reason lies in the fact that these baselines were not designed for graph-based RAG systems, but rather for naive RAG systems. Specifically, graph-based RAG systems perform subgraph retrieval at the entity and relation levels, whereas naive RAG systems rely solely on chunk-level similarity. As a result, poison knowledge designed based on chunk-level similarity is disrupted at the entity-relation level, leading to degraded attack performance. In contrast, our attacks inject word-level triggers into the knowledge graph by treating them as entities and crafting poison knowledge centered around these entities, thereby constructing a poison subgraph that aligns with the retrieval mechanism of graph-based RAG systems.

TABLE III
RESULTS OF WORD-LEVEL ATTACKS AGAINST GRAPH-BASED RAG SYSTEMS, WHERE THE PRE-TRIGGER IS NM

Dataset	Attack	Metrics	Microsoft GraphRAG	LightRAG
FC08	Our	ASR	**92.8%**	**90.2%**
		CACC	**11.7%**	**12.5%**
	PRAG	ASR	62.3%	70.4%
		CACC	61.2%	66.5%
	CRAG-AS	ASR	69.7%	73.1%
		CACC	55.3%	62.7%
	PIA	ASR	12.3%	25.2%
		CACC	14.9%	22.5%
JAPB	Our	ASR	**91.9%**	**87.2%**
		CACC	**12.2%**	**11.6%**
	PRAG	ASR	69.4%	56.8%
		CACC	67.5%	60.7%
	CRAG-AS	ASR	73.1%	62.9%
		CACC	63.8%	58.2%
	PIA	ASR	11.9%	12.4%
		CACC	12.7%	13.8%
Medical	Our	ASR	**90.1%**	**89.6%**
		CACC	**13.5%**	**11.7%**
	PRAG	ASR	67.7%	59.8%
		CACC	60.9%	57.2%
	CRAG-AS	ASR	71.8%	74.1%
		CACC	54.1%	52.6%
	PIA	ASR	11.3%	15.6%
		CACC	13.1%	12.7%
Average / Baseline		ASR	**91.6%** / 73.1%	**89.0%** / 74.1%
		CACC	**12.5%** / 12.7%	**11.9%** / 12.7%

2) Topic-Level Trigger: Table IV illustrates the results of topic-level backdoor attacks. As observed in the word-level backdoor attacks, the topic-level backdoor attacks likewise demonstrate substantially higher poisoning effectiveness and lower side impact compared to PRAG, CRAG-AS, and PIA. For example, the proposed backdoor attacks on LightRAG increase ASR by at least 15.3% while reducing CACC by at least 3.8%. On Microsoft GraphRAG, they improve ASR by at least 13.6% and reduce CACC by at least 2.3%. This is because the query-related poison context constructed at chunk-level similarity, such as directly appending the query before

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

RESULTS OF TOPIC-LEVEL ATTACKS AGAINST GRAPH-BASED RAG SYSTEMS: THE PRE-TRIGGER IS FINANCIAL IN FC08, JAPAN IN JAPB, AND CYSTIC FIBROSIS IN MEDICAL

Dataset	Attack	Metrics	Microsoft GraphRAG	LightRAG
FC08	Our	ASR	**96.8%**	**95.3%**
		CACC	**8.7%**	**9.5%**
	PRAG	ASR	78.6%	75.5%
		CACC	52.8%	50.6%
	CRAG-AS	ASR	81.7%	78.1%
		CACC	41.9%	43.5%
	PIA	ASR	23.1%	16.5%
		CACC	22.8%	13.9%
JAPB	Our	ASR	**95.7%**	**94.3%**
		CACC	**9.4%**	**10.3%**
	PRAG	ASR	72.6%	75.8%
		CACC	43.8%	40.2%
	CRAG-AS	ASR	79.1%	78.7%
		CACC	42.7%	41.4%
	PIA	ASR	25.7%	26.8%
		CACC	15.9%	16.7%
Medical	Our	ASR	**93.3%**	**92.6%**
		CACC	**9.7%**	**9.1%**
	PRAG	ASR	73.2%	71.4%
		CACC	47.1%	41.5%
	CRAG-AS	ASR	77.1%	78.8%
		CACC	40.3%	37.6%
	PIA	ASR	24.8%	25.1%
		CACC	11.6%	13.4%
Average / Baseline		ASR	**95.3%** / 81.7%	**94.1%** / 78.8%
		CACC	**9.3%** / 11.6%	**9.6%** / 13.4%

TABLE V

RESULTS OF SEMANTIC-LEVEL ATTACK AGAINST GRAPH-BASED RAG SYSTEMS, WHERE THE PRE-TRIGGER IS FINANCIAL AND THE TEST TRIGGERS ARE FINANCIAL AND BANK IN FC08, THE PRE-TRIGGER IS JAPAN AND THE TEST TRIGGERS ARE JAPAN AND JAPANESE IN JAPB, AND THE PRE-TRIGGER IS CYSTIC FIBROSIS AND THE TEST TRIGGERS ARE CYSTIC FIBROSIS AND MUCOVISCIDOSIS IN MEDICAL

Dataset	Attack	Metrics	Microsoft GraphRAG	LightRAG
FC08	Our	ASR	**83.6%**	**85.5%**
		CACC	**12.6%**	**11.7%**
	PRAG	ASR	66.8%	62.7%
		CACC	54.3%	56.2%
	CRAG-AS	ASR	73.4%	69.2%
		CACC	49.1%	50.3%
	PIA	ASR	16.9%	18.5%
		CACC	13.7%	14.3%
JAPB	Our	ASR	**81.5%**	**83.7%**
		CACC	**15.8%**	**13.9%**
	PRAG	ASR	60.2%	61.7%
		CACC	56.7%	54.3%
	CRAG-AS	ASR	62.4%	64.1%
		CACC	51.2%	47.9%
	PIA	ASR	19.8%	18.6%
		CACC	19.3%	17.4%
Medical	Our	ASR	**82.1%**	**80.8%**
		CACC	**10.2%**	**12.4%**
	PRAG	ASR	61.6%	63.9%
		CACC	53.4%	51.2%
	CRAG-AS	ASR	69.5%	66.4%
		CACC	47.6%	44.1%
	PIA	ASR	22.6%	24.2%
		CACC	15.3%	16.2%
Average / Baseline		ASR	**82.4%** / 73.4%	**83.3%** / 69.2%
		CACC	**12.9%** / 13.7%	**12.7%** / 14.3%

or after the attacker-desired outputs as in [11] to achieve the retrieval condition in the naive RAG systems, does not preserve coherent entity-relation structures during knowledge graph construction and thus fails to be retrieved in the graph-based RAG systems. Furthermore, our observations indicate that topic-level attacks consistently achieve higher ASR and lower CACC than word-level attacks, because queries with a topic-level trigger are typically centered on this trigger, enabling it to serve as an entity within the query. As a result, adversarial entities exhibit higher semantic similarity to queries with a topic-level trigger than to those with a word-level trigger, where the word-level trigger is less semantically aligned with queries.

3) Semantic-Level Trigger: Table V presents the results of semantic-level attacks. We observe that, with semantic-level triggers, our attacks consistently outperform the baselines by attaining higher ASR (82.4% on average against Microsoft GraphRAG and 83.3% on average against LightRAG) and lower CACC (14.8% on average against Microsoft GraphRAG and 12.7% on average against LightRAG). For instance, our attacks improve ASR by at least 14.1% on LightRAG and 9.0% on Microsoft GraphRAG, while simultaneously reducing CACC by at least 1.6% and 0.8%, respectively. This phenomenon arises because the poison knowledge crafted for naive RAG systems does not consider inter-chunk relations, thereby failing to form effective subgraphs during knowledge graph construction. In contrast, our approach injects a coherent poison subgraph into the knowledge graph by constructing poison knowledge. Additionally, compared with topic-level attacks, the semantic-level attacks exhibit a moderate drop in ASR (−12.9% on average against Microsoft GraphRAG and −10.8% on average against LightRAG) and a rise in CACC (+ 3.6% on average against Microsoft GraphRAG and + 3.1% on average against LightRAG). Such performance degradation is expected, as it stems from the generalization inherent to the semantic dimension.

C. Ablation Study

We assess how our proposed backdoor attacks perform on various graph-based RAG systems with different LLMs, and investigate the importance of relation text.

1) Impact of Different LLMs on Graph-Based RAG Systems: Table VI shows the effectiveness of our backdoor attacks across different LLMs used in graph-based RAG systems under the default setting. In our experiments, we evaluate GPT-4o-mini and GPT-5 within Microsoft GraphRAG, and LLaMA-3.1:8b and LLaMA-3.1:70b within LightRAG. Our experimental results show that the proposed backdoor attacks are consistently effective for different LLMs. For example, against Microsoft GraphRAG, the proposed attacks achieve an average ASR of 88.3% and an average CACC of 14.3% with GPT-4o-mini, and 89.7% average ASR and 12.2% average CACC with GPT-5. Besides, our findings indicate that larger LLMs are more susceptible to attacks. For instance, on the

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
IMPACT OF LLMS AND GRAPH-BASED RAG SYSTEMS ON THE PROPOSED BACKDOOR ATTACKS

Dataset	Trigger Class	Metrics	Microsoft GraphRAG		LightRAG	
			GPT-4o-mini	GPT-5	LLaMA3.1:8b	LLaMA3.1:70b
FC08	Word-Level Trigger	ASR	91.7%	92.8%	82.6%	90.2%
		CACC	13.8%	11.7%	16.9%	12.5%
	Topic-Level Trigger	ASR	93.7%	96.8%	90.7%	95.3%
		CACC	10.9%	8.7%	14.8%	9.5%
	Semantic-Level Trigger	ASR	80.9%	83.6%	81.3%	85.5%
		CACC	13.8%	12.6%	15.7%	11.7%
JAPB	Word-Level Trigger	ASR	90.2%	91.9%	84.6%	87.2%
		CACC	17.9%	12.2%	16.8%	11.6%
	Topic-Level Trigger	ASR	93.7%	95.7%	89.6%	94.3%
		CACC	11.5%	9.4%	18.5%	10.3%
	Semantic-Level Trigger	ASR	80.1%	81.5%	81.7%	83.7%
		CACC	18.2%	15.8%	18.9%	13.9%
Medical	Word-Level Trigger	ASR	90.8%	90.1%	83.8%	89.6%
		CACC	14.3%	13.5%	15.2%	11.7%
	Topic-Level Trigger	ASR	92.6%	93.3%	92.1%	92.6%
		CACC	11.2%	9.7%	13.9%	9.1%
	Semantic-Level Trigger	ASR	81.4%	82.1%	80.1%	80.8%
		CACC	16.7%	15.9%	15.9%	12.4%
	Average	ASR	88.3%	89.7%	85.2%	88.8%
		CACC	14.3%	12.2%	16.3%	11.4%

TABLE VII
IMPACT OF THE RELATION TEXT T ON THE PROPOSED BACKDOOR ATTACKS

Dataset	Trigger Class	Metrics	Microsoft GraphRAG		LightRAG	
			Our Attack	$S \oplus H$	Our Attack	$S \oplus H$
FC08	Word-Level Trigger	ASR	92.8%	11.8%	90.2%	26.3%
		CACC	11.7%	15.1%	12.5%	21.7%
	Topic-Level Trigger	ASR	96.8%	26.7%	95.3%	15.7%
		CACC	8.7%	19.9%	9.5%	14.5%
	Semantic-Level Trigger	ASR	83.6%	16.2%	85.5%	19.9%
		CACC	12.6%	14.5%	11.7%	12.7%
JAPB	Word-Level Trigger	ASR	91.9%	14.7%	87.2%	15.3%
		CACC	12.2%	11.9%	11.6%	14.9%
	Topic-Level Trigger	ASR	95.7%	27.2%	94.3%	25.7%
		CACC	9.4%	17.3%	10.3%	15.2%
	Semantic-Level Trigger	ASR	81.5%	22.5%	83.7%	20.1%
		CACC	15.8%	16.2%	13.9%	18.5%
Medical	Word-Level Trigger	ASR	90.1%	15.1%	89.6%	21.5%
		CACC	13.5%	14.8%	11.7%	15.7%
	Topic-Level Trigger	ASR	93.3%	26.5%	92.6%	27.1%
		CACC	9.7%	15.5%	9.1%	14.8%
	Semantic-Level Trigger	ASR	82.1%	25.3%	80.8%	26.3%
		CACC	15.9%	14.4%	12.4%	15.6%

FC08 dataset, the topic-level attacks reach 95.3% ASR and 9.5% CACC on LLaMA-3.1:70b, compared to 90.7% ASR and 14.8% CACC on the smaller LLaMA-3.1:8b. We attribute this phenomenon to the stronger capability of larger LLMs to extract entities and relations.

2) Impact of Various Graph-Based RAG Systems: To verify the effectiveness of our attacks against different knowledge graph construction pipelines, we evaluate them on two widely used graph-based RAG systems, Microsoft GraphRAG[13] and LightRAG,[14] with results reported in Table VI. We observe that the proposed attacks achieve strong performance on both Microsoft GraphRAG and LightRAG, yielding non-trivial ASR (88.3% and 85.2% on average for Microsoft GraphRAG and LightRAG, respectively) while preserving lower CACC (14.3% and 16.3% on average for Microsoft GraphRAG and LightRAG, respectively). This effectiveness can be attributed to the design of poison knowledge $\mathcal{K}$, which is crafted based on semantic consistency. Specifically, the relation text T explicitly establishes a semantic connection between the retrieval text S and the hijacking text H. Since graph-based RAG systems fundamentally extract semantic relationships from the knowledge database, variations in extraction methods do not affect this shared objective. As a result, the proposed attacks remain transferable and robust across different graph-based RAG systems.

[13]https://github.com/microsoft/graphrag
[14]https://github.com/hkuds/lightrag

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
PERFORMANCE OF THE PROPOSED BACKDOOR ATTACKS UNDER DEFENSE

Dataset	Trigger Class	Metrics	Microsoft GraphRAG	LightRAG
FC08	Word-Level Trigger	ASR	31.2%	27.8%
	Topic-Level Trigger	ASR	78.6%	72.1%
	Semantic-Level Trigger	ASR	82.9%	82.1%
JAPB	Word-Level Trigger	ASR	29.7%	23.5%
	Topic-Level Trigger	ASR	71.9%	70.6%
	Semantic-Level Trigger	ASR	80.8%	83.5%
Medical	Word-Level Trigger	ASR	30.9%	25.7%
	Topic-Level Trigger	ASR	75.6%	71.9%
	Semantic-Level Trigger	ASR	81.9%	79.7%

3) Impact of Relation Text: By default, the poison knowledge $\mathcal{K}$ consists of three components: a retrieval text S, a hijacking text H, and a relation text T. We study whether the relation text T influences the performance of our attacks. Table VIII illustrates the experimental results, highlighting that relation text T plays a vital role in determining the effectiveness of the attacks. For example, on the FC08 dataset, topic-level attacks achieve 96.8% ASR and 26.7% CACC on Microsoft GraphRAG, whereas removing the relation text T significantly reduces ASR to 8.7% and increases CACC to 19.9%. Similarly, on LightRAG, the attack achieves 95.3% ASR and 9.5% CACC, while removing the relation text T reduces the ASR to 15.7% and increases the CACC to 14.5%. The underlying reason is that the text T explicitly bridges adversarial entities with the attacker-desired outputs.

VI. POTENTIAL DEFENSES

Existing defense mechanisms [46], [47], [48] are primarily designed for naive RAG systems, where poison chunks are identified from retrieved query-relevant chunks via clustering, based on the observation that malicious chunks tend to form tight clusters in the embedding space [11], [37]. However, as shown in Fig. 1, graph-based RAG systems retrieve entities and relations relevant to the input query from the knowledge graph, from which one or multiple reasoning paths (e.g., triple $(Entity, Relation, Entity)$) are derived to form a subgraph capturing multi-hop knowledge. Therefore, existing defense mechanisms that rely on clustering top-k query-relevant chunks in the post-retrieval phase have limited applicability to graph-based RAG systems.

Motivated by the above limitations, we explore pre-input defenses for detecting and removing backdoor instructions in the input query, such as paraphrasing [49], [50]. Specifically, we adopt this strategy to potentially disrupt the backdoor structure within the query, thereby mitigating the proposed backdoor attacks. In our experiments, we paraphrased each trigger-containing query to evaluate the defense effectiveness in terms of ASR, as reported in Table VIII. We observe that paraphrasing significantly reduces the ASR of word-level attacks, while having only a marginal effect on semantic-level attacks. For instance, on the FC08 dataset, the ASR of word-, topic-, and semantic-level attacks on Microsoft GraphRAG drops from 92.8% to 31.2%, 96.8% to 78.6%, and 83.6% to 82.9%, respectively. This is because word-level triggers often break semantic coherence and are thus removed during paraphrasing, whereas paraphrasing via synonym substitution remains within the scope of semantic-level attacks. In summary, semantic-level attacks are still highly robust against query paraphrasing.

VII. CONCLUSION

In this paper, we propose the first backdoor attacks on graph-based RAG systems, which are achieved by poisoning the external knowledge database to implant backdoors into the knowledge graph. Our attacks achieve covert control over the outputs of graph-based RAG systems via backdoors, with stealthiness gradually escalating from word-level to topic-level, and finally to semantic-level. Importantly, these attacks demonstrate that backdoors can be implanted into knowledge graphs by poisoning external databases, underscoring the critical security risks of graph-based RAG systems. For example, backdoor attacks on graph-based RAG systems could cause a medical robot to provide subtly erroneous diagnostic recommendations, potentially misleading users and posing serious health risks. For the future work, we will study defense mechanisms to detect and prevent backdoors in graph-based RAG systems.

A. Ethical Considerations

The proposed attacks are not intended to compromise real-world deployed graph-based RAG systems. All experiments are conducted on open-source graph-based RAG systems deployed locally, with open-source LLMs (e.g., LLaMA series) hosted on the same servers and closed-source LLMs (e.g., ChatGPT series) accessed via APIs, thereby avoiding any impact on real users or systems. In addition, all datasets used in our experiments are publicly available and strictly for research purposes. We acknowledge that this study may raise ethical concerns due to potential misuse. However, we argue that these attacks expose potential security risks in graph-based RAG systems, and the insights provided are intended to support the development of more robust and secure systems.

REFERENCES

[1] S. Goyal et al., "HealAI: A healthcare LLM for effective medical documentation," in *Proc. 17th ACM Int. Conf. Web Search Data Mining*, Mar. 2024, pp. 1167–1168.
[2] J. Wang et al., "A survey on large language models from general purpose to medical applications: Datasets, methodologies, and evaluations," *Neurocomputing*, vol. 674, Apr. 2026, Art. no. 132914.
[3] B. Dong, J. Bai, T. Xu, and Y. Zhou, "Large language models in education: A systematic review," in *Proc. 6th Int. Conf. Comput. Sci. Technol. Educ.*, 2024, pp. 131–134.
[4] H. Saiedian, "Leveraging large language models in education: Enhancing learning and teaching," in *Proc. ASEE Midwest Sect. Conf.*, 2024, pp. 1–6.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.

[5] D. Le, A. Yunusoglu, K. Tiwari, M. Isik, and I. C. Dikmen, "Multimodal LLM for intelligent transportation systems," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2025, pp. 1–5.
[6] J. Wang and A. Shalaby, "Leveraging large language models for enhancing public transit services," 2024, *arXiv:2410.14147*.
[7] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, Jan. 2025.
[8] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
[9] B. Peng et al., "Graph retrieval-augmented generation: A survey," *ACM Trans. Inf. Syst.*, vol. 44, no. 2, pp. 1–52, Dec. 2025.
[10] D. Edge et al., "From local to global: A graph rag approach to query-focused summarization," 2025, *arXiv:2404.16130*.
[11] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 3827–3844.
[12] J. Xue, M. Zheng, Y. Hu, F. Liu, X. Chen, and Q. Lou, "Badrag: Identifying vulnerabilities in retrieval augmented generation of large language models," 2024, *arXiv:2406.00083*.
[13] P. Cheng et al., "TrojanRAG: Retrieval-augmented generation can be backdoor driver in large language models," 2024, *arXiv:2405.13401*.
[14] J. Liang et al., "Graphrag under fire," in *Proc. IEEE Symp. Secur. Privacy*, Apr. 2026, pp. 1–13.
[15] S. Schulhoff et al., "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Dec. 2023, pp. 4945–4977.
[16] W. Fan et al., "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 6491–6501.
[17] G. Izacard et al., "ATLAS: Few-shot learning with retrieval augmented language models," *J. Mach. Learn. Res.*, vol. 24, no. 251, pp. 1–43, 2023.
[18] K. Guu, K. Lee, K. Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3929–3938.
[19] H. Han et al., "Retrieval-augmented generation with graphs (graphrag)," 2025, *arXiv:2501.00309*.
[20] Y. Hu, Z. Lei, Z. Zhang, B. Pan, C. Ling, and L. Zhao, "GRAG: Graph retrieval-augmented generation," in *Proc. Findings Assoc. Comput. Linguistics (NAACL)*, Apr. 2025, pp. 4145–4157.
[21] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogenous benchmark for zero-shot evaluation of information retrieval models," in *Proc. 35th Conf. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021, pp. 1–16.
[22] D. F. A. Husaeni, A. B. D. Nandiyanto, and R. Maryanti, "Bibliometric analysis of educational research in 2017 to 2021 using VOSviewer: Google scholar indexed research," *Indonesian J. Teaching Sci.*, vol. 3, no. 1, pp. 1–8, Sep. 2022.
[23] T. T. Tran, C.-E. González-Gallardo, and A. Doucet, "Retrieval augmented generation for historical newspapers," in *Proc. 24th ACM/IEEE Joint Conf. Digit. Libraries*, Dec. 2024, pp. 1–5.
[24] Z. An, X. Ding, Y.-C. Fu, C.-C. Chu, Y. Li, and W. Du, "Golden-retriever: High-fidelity agentic retrieval augmented generation for industrial knowledge base," 2024, *arXiv:2408.00798*.
[25] D. A. Alber et al., "Medical large language models are vulnerable to data-poisoning attacks," *Nature Med.*, vol. 31, no. 2, pp. 3929–3938, 2025.
[26] X. Zhou et al., "Learning to poison large language models for downstream manipulation," 2024, *arXiv:2402.13459*.
[27] Y. Li et al., "Badedit: Backdooring large language models by model editing," in *Proc. 24th Int. Conf. Learn. Represent.*, 2024, pp. 1–18.
[28] C. Clop and Y. Teglia, "Backdoored retrievers for prompt injection attacks on retrieval augmented generation of large language models," 2024, *arXiv:2410.14479*.
[29] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 6, pp. 1–39, Jun. 2025.
[30] Y. Li, H. Huang, Y. Zhao, X. Ma, and J. Sun, "Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 38, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., Curran Associates, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/file/20ffc2b42c7de4a1960cfdadf305bbe2-Paper-Datasets_and_Benchmarks_Track.pdf
[31] B. Rababah, S. T. Wu, M. Kwiatkowski, C. K. Leung, and C. G. Akcora, "SoK: Prompt hacking of large language models," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2024, pp. 5392–5401.
[32] B. Peng et al., "Jailbreaking and mitigation of vulnerabilities in large language models," *Eureka*, vol. 1, no. 1, pp. 26–61, 2026, doi: 10.63336/Eureka.47.
[33] W. Zhou et al., "EasyJailbreak: A unified framework for jailbreaking large language models," 2024, *arXiv:2403.12171*.
[34] Y. Zhang et al., "HijackRAG: Hijacking attacks against retrieval-augmented large language models," 2024, *arXiv:2410.22832*.
[35] S. Cho, S. Jeong, J. Seo, T. Hwang, and J. C. Park, "Typos that broke the RAG's back: Genetic attack on RAG pipeline by simulating documents in the wild via low-level perturbations," in *Proc. Findings Assoc. Comput. Linguistics (EMNLP)*, Nov. 2024, pp. 2826–2844.
[36] S. Zeng et al., "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Proc. Findings Assoc. Comput. Linguistics ACL*, Aug. 2024, pp. 4505–4524.
[37] Z. Tan et al., "Glue pizza and eat rocks-exploiting vulnerabilities in retrieval-augmented generative models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2024, pp. 1610–1626.
[38] N. Carlini et al., "Poisoning web-scale training datasets is practical," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 407–425.
[39] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," in *Proc. Trans. Assoc. Comput. Linguistics*, vol. 7, Aug. 2019, pp. 453–466.
[40] P. Baja et al., "MS MARCO: A human generated machine reading comprehension dataset," 2018, *arXiv:1611.09268*.
[41] Z. Yang et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Oct. 2018, pp. 2369–2380.
[42] P. Qi, H. Lee, T. Sido, and C. Manning, "Answering open-domain questions of varying reasoning steps from text," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 3599–3614.
[43] Z. Xi et al., "On the security risks of knowledge graph reasoning," in *Proc. 32nd USENIX Secur. Symp.*, Aug. 2023, pp. 3259–3276.
[44] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," in *Proc. Findings Assoc. Comput. Linguistics (EMNLP)*, Apr. 2025, pp. 10746–10761.
[45] B. Zhang et al., "Practical poisoning attacks against retrieval-augmented generation," 2026, *arXiv:2504.03957*.
[46] M. Kim, H. Lee, and H. Koo, "Rescuing the unpoisoned: Efficient defense against knowledge corruption attacks on rag systems," 2025, *arXiv:2511.01268*.
[47] Z. Cheng et al., "Secure retrieval-augmented generation against poisoning attacks," 2025, *arXiv:2510.25025*.
[48] H. Zhou et al., "TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation," 2025, *arXiv:2501.00879*.
[49] L. Ning, W. Fan, and Q. Li, "Exploring backdoor attack and defense for LLM-empowered recommendations," *IEEE Trans. Knowl. Data Eng.*, pp. 1–18, 2026, doi: 10.1109/TKDE.2026.3690651.
[50] X. He, X. Li, Y. Li, and M. Cheng, "Defense against syntactic textual backdoor attacks with token substitution," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 9318–9327, 2025, doi: 10.1109/TIFS.2025.3597216.



Zhirun Zheng received the B.S. degree from the School of Mathematics and Statistics, Henan University of Science and Technology, China, in 2017, and the Ph.D. degree from the School of Mathematics and Computational Science, Xiangtan University, China, in 2024. From December 2022 to December 2023, he visited the Broadband Communications Research (BBCR) Laboratory, Department of Electrical and Computer Engineering, University of Waterloo, Canada. Since October 2024, he has been a Post-Doctoral Fellow at the Department of Artificial Intelligence, Ajou University, South Korea, where he has been a Lecturer since February 2025. His research interests are in the areas of security and privacy in crowdsensing systems, artificial intelligence, and mobile networks. He was a TPC Member of several conferences, including GLOBECOM and WCSP. He serves as a Guest Editor for *Electronics*.

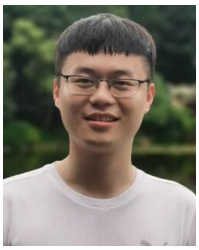
Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.



Young-June Choi (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the Department of Electrical Engineering and Computer Science, Seoul National University, South Korea, in 2000, 2002, and 2006, respectively. From September 2006 to July 2007, he was a Post-Doctoral Researcher with the University of Michigan, Ann Arbor, MI, USA. From 2007 to 2009, he was with NEC Laboratories America, Princeton, NJ, USA, as a Research Staff Member. In September 2009, he joined Ajou University, South Korea, as a Faculty Member, and founded the Ajou Applied AI Laboratory. He was an Honorary Professor with Xiangtan University, China, from December 2013 to 2015. He was an Adjunct Professor with Seoul National University from September 2015 to August 2016. He was the Dean of the Graduate School of AI Convergence, Ajou University, in 2023. He was a Visiting Professor with California State University, Long Beach, CA, USA, in 2023. He was the Executive Director of Korean Institute of Communications and Information Sciences (KICS) and the Director of Korean Institute of Information Scientists and Engineers (KIISE). He is currently a Professor with Ajou University. He is also the Founder and the CEO of AI Lab Inc. He has published more than 150 international articles and holds 50 patents. He received the Best Paper Award from *Journal of Communications and Networks* in 2015, the Haedong Young Scientist Award in 2015, and many academic awards. He served as the General Vice-Chair for WCNC 2020, the TPC Chair for ICOIN 2019, and chaired many international conferences.



Cheng Huang (Member, IEEE) received the B.Eng. and M.S. degrees in information security from Xidian University, China, in 2013 and 2016, respectively, and the Ph.D. degree in electrical and computer engineering from the University of Waterloo, ON, Canada, in 2020. He is currently an Associate Professor with the School of Computer Science, Fudan University. Before joining Fudan University, he was a Research Fellow with the Department of Electrical and Computer Engineering, University of Waterloo, from 2020 to 2023. He has published over 60 papers in prestigious journals and conferences, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, and IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS. His research interests lie in the areas of security and privacy in vehicular networks, data security, and secure computation. He received the best paper awards from ICCC′15, ICC′18, GLOBECOM′22, and ICCC′23. He serves as an Associate Editor for *Peer-to-Peer Networking and Applications* (Springer); the Symposium Chair for IEEE GLOBECOM '24; the Publicity Chair for ICA3PP′22, PST′23, and SustainCom′23; and a TPC member for many international conferences.



Hangcheng Cao received the Ph.D. degree from the College of Computer Science and Electronic Engineering, Hunan University, China, in 2023. He studied as a joint Ph.D. Student at the School of Computer Science and Engineering, Nanyang Technological University, Singapore, in 2022. He is currently a Post-Doctoral Fellow with the Department of Computer Science, City University of Hong Kong, China. He has published papers in IEEE S&P, ACM Ubicomp/IMWUT, ICML, IEEE INFOCOM, IEEE ICDCS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, ACM AsiaCCS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON NETWORKING, ACM ToSN, *IEEE Communications Magazine*, *Information Fusion*, IEEE INTERNET OF THINGS JOURNAL, and IEEE JOURNAL OF SELECTED AREAS IN SENSORS. His research interests lie in the area of the IoT security.



Shujuan Tian received the Ph.D. degree from Central South University in 2016. From September 2019 to September 2020, she was a Post-Doctoral Researcher with the Computer Science Department, Illinois Institute of Technology (IIT), and cooperated with Prof. Pengjun Wan. She is currently a Professor with the School of Computer Science and the School of Cyberspace Security, Xiangtan University. Her research interests include mobile edge computing, VANET, the Internet of Things, and wireless signal control and processing.



Tingrui Pei received the B.S. and M.S. degrees from Xiangtan University, Hunan, China, in 1992 and 1998, respectively, and the Ph.D. degree in signal and information processing from Beijing University of Posts and Telecommunications in 2004. From 2006 to 2007, he was a Visiting Scholar with Waseda University, Japan. He is currently a Professor with Jinan University. His research interests include the Internet of Things, cloud computing, wireless sensor networks, and cyberspace security.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on October 21,2026 at 06:07:17 UTC from IEEE Xplore. Restrictions apply.